

COMSATS UNIVERSITY ISLAMABAD



OBJECT ORIENTED PROGRAMMING **PROJECT REPORT**

NAME:

- AINA AZHAR (SP24-BDS-003)
- ALI HAMZA (SP25-BCT-004)

SECTION: BDS-4A & BCT-2A

SUBMITTED TO: Sir Yasir Munir

COMSATS UNIVERSITY ISLAMABAD

Contents

INTRODUCTION:	3
TASK DISTRIBUTION:	3
MUHAMMAD AZLAN	Error! Bookmark not defined.
SAHAR ZAHID	Error! Bookmark not defined.
MUSA SAJJAD	Error! Bookmark not defined.
ATTRIBUTES AND ENTITIES:	4
1. DashboardScreen (JFrame):	Error! Bookmark not defined.
2. ExportScreen:	Error! Bookmark not defined.
3. FeedbackAnalyzer:	Error! Bookmark not defined.
4. FreeUser:	5
5. HistoryScreen:	Error! Bookmark not defined.
6. Leaderboard:	Error! Bookmark not defined.
7. LeaderboardScreen:	Error! Bookmark not defined.
8. LoginScreen:	6
9. PremiumUser:	6
10. Result:	6
11. ResultScreen:	7
12. TextGenerator:	7
13. TypingApp:	7
14. TypingBehavior (Interface):	7
15. TypingTest:	7
16. TypingTestGUI:	8
17. User (Abstract Class):	8
FEATURES OF SYSTEM:	10
1. Login Module:	10
2. Typing Test Module:	10
3. Feedback and Result Module:	10
4. User and Leaderboard Module:	11
5. Text Generator Module:	11
6. GUI and Event Handling Module:	12
PROJECT WALKTHROUGH:	Error! Bookmark not defined.

COMSATS UNIVERSITY ISLAMABAD

TYPEMASTER: A Real-Time Typing Speed Evaluation System

INTRODUCTION:

The **Typing Speed Application** is a desktop-based Java application designed to test and enhance a user's typing skills. Built using Java Swing and Object-Oriented Programming (OOP) principles, this project provides an engaging user interface for measuring typing speed (WPM), accuracy, and mistake count in real time. The system includes support for both free and premium users, offers difficulty-based text generation, and stores user performance data in a leaderboard.

This project demonstrates core OOP concepts such as inheritance, abstraction, interfaces, exception handling, file handling, serialization, and modular design with GUI integration.

TASK DISTRIBUTION:

ALI HAMZA

- Designed the User, FreeUser, and PremiumUser classes using inheritance and abstract classes.
- Implemented the complete login system with GUI (LoginScreen) including registration.
- Handled user serialization for login persistence (users.dat file).
- Created DashboardScreen with role-based navigation and premium feature detection.
- Developed HistoryScreen for displaying user-specific typing history.
- Built ResultScreen to show final test results with navigation back to dashboard.
- Implemented all navigation logic between different GUI screens.
- Managed user authentication, validation, and account type switching.

AINA AZHAR

- Developed the core typing engine in TypingTest class with difficulty selection.
- Created TypingTestGUI with real-time character highlighting and keyboard input handling.

COMSATS UNIVERSITY ISLAMABAD

- Implemented FeedbackAnalyzer to calculate WPM, accuracy, and mistake count.
- Designed TextGenerator to provide difficulty-based random text from files.
- Built the Leaderboard system using Java Generics for result storage and retrieval.
- Created LeaderboardScreen to display global rankings of top performers.
- Developed ExportScreen for premium users to save history as text files.
- Implemented binary serialization for results persistence (results.dat file).
- Handled all file I/O operations including text file reading and error management.
- Managed real-time statistics tracking during typing tests.

ATTRIBUTES AND METHODS:

1. DashboardScreen (JFrame)

Attributes:

- currentUser: User - Stores the logged-in user
- leaderboard: Leaderboard<Result> - Leaderboard instance

Methods:

- DashboardScreen(User, Leaderboard<Result>) - Constructor
- Action listeners for buttons (startTest, viewLeaderboard, viewHistory, exportHistory)

2. ExportScreen (JFrame)

Attributes:

- leaderboard: Leaderboard<Result>
- username: String

Methods:

- ExportScreen(Leaderboard<Result>, String) - Constructor
- exportData() - Handles file export

COMSATS UNIVERSITY ISLAMABAD

- exportButton action listener

3. FeedbackAnalyzer

Attributes:

- wpm: double
- accuracy: double
- mistakes: int

Methods:

- evaluatePerformance(String, String, double) - Calculates performance metrics
- getWPM(): double
- getAccuracy(): double
- getMistakes(): int

4. FreeUser (extends User)

Methods:

- FreeUser(String, String) - Constructor
- typingMode() - Implementation
- isPremium(): boolean - Returns false

5. HistoryScreen (JFrame)

Attributes:

- historyArea: JTextArea
- leaderboard: Leaderboard<Result>
- username: String

Methods:

- HistoryScreen(Leaderboard<Result>, String) - Constructor
- updateHistoryDisplay() - Updates the display

6. Leaderboard<T extends Result>

Attributes:

- results: ArrayList<T>
- fileName: String (constant) - "results.dat"

COMSATS UNIVERSITY ISLAMABAD

Methods:

- Leaderboard() - Constructor
- addResult(T)
- getTopResults(int): List<T>
- getUserHistory(String): List<T>
- saveResults() - Private
- loadResults() - Private

7. LeaderboardScreen (JFrame)

Attributes:

- leaderboardArea: JTextArea
- leaderboard: Leaderboard<Result>
- TOP_N: int (constant) - 10

Methods:

- LeaderboardScreen(Leaderboard<Result>) - Constructor
- updateLeaderboardDisplay()

8. LoginScreen (JFrame)

Attributes:

- usernameField: JTextField
- passwordField: JPasswordField
- users: ArrayList<User> (static)
- usersFileName: String (constant) - "users.dat"
- globalLeaderboard: Leaderboard<Result>

Methods:

- LoginScreen(Leaderboard<Result>) - Constructor
- login()
- register()
- saveUsers()
- loadUsers()

9. PremiumUser (extends User)

COMSATS UNIVERSITY ISLAMABAD

Methods:

- PremiumUser(String, String) - Constructor
- typingMode() - Implementation
- isPremium(): boolean - Returns true

10. Result (implements Serializable)

Attributes:

- username: String
- wpm: double
- accuracy: double
- mistakes: int

Methods:

- Result(String, double, double, int) - Constructor
- Getters: getUsername(), getWpm(), getAccuracy(), getMistakes()
- toString(): String

11. ResultScreen (JFrame)

Methods:

- ResultScreen(double, double, int, User, Leaderboard<Result>) - Constructor
- Action listener for backToDashboardButton

12. TextGenerator

Attributes:

- easy: ArrayList<String>
- medium: ArrayList<String>
- hard: ArrayList<String>
- difficultyLevel: String

Methods:

- TextGenerator() - Constructor
- loadText()
- loadFromFile(String, ArrayList<String>)
- setDifficulty(int)
- getRandomText(): String

COMSATS UNIVERSITY ISLAMABAD

13. TypingApp (Main class)

Methods:

- main(String[]) - Entry point

14. TypingBehavior (Interface)

Methods:

- typingMode() - Abstract method

15. TypingTest (implements TypingTestGUI, TypingTestListener)

Attributes:

- user: User
- textGenerator: TextGenerator
- feedbackAnalyzer: FeedbackAnalyzer
- totalTime: double
- originalText: String
- typedText: String
- leaderboard: Leaderboard<Result>

Methods:

- TypingTest(User, Leaderboard<Result>) - Constructor
- selectLevelAndGetText()
- getText()
- startTypingTest()
- onTypingComplete(String, double) - Interface implementation
- storeResult()
- displayResult()

16. TypingTestGUI

Attributes:

- startTime: long
- endTime: long
- started: boolean
- typedText: StringBuilder
- currentIndex: int

COMSATS UNIVERSITY ISLAMABAD

- timer: Timer
- timerLabel: JLabel
- textPane: JTextPane

Methods:

- showGUI(String, TypingTestListener)
- updateStatsDisplay(String, String, double)
- completeTest(String, TypingTestListener, JFrame)

Interface:

- TypingTestListener with method onTypingComplete(String, double)

17. User (abstract, implements TypingBehavior, Serializable)

Attributes:

- username: String
- password: String

Methods:

- User(String, String) - Constructor
- getUsername(): String
- checkPassword(String): boolean
- isPremium(): boolean - Abstract
- typingMode() - Abstract (from TypingBehavior)

Text Files:

- easy.txt - Contains easy difficulty texts
- medium.txt - Contains medium difficulty texts
- hard.txt - Contains hard difficulty texts

COMSATS UNIVERSITY ISLAMABAD

FEATURES OF SYSTEM:

1. Login Module (4 Features)

1. User Authentication

- Validates username and password against stored user data
- Handles both free and premium user logins

2. User Registration

- Creates new user accounts (both free and premium types)
- Validates username uniqueness
- Stores credentials securely

3. Account Type Selection

- Offers premium/free account options during registration
- Differentiates features based on account type

4. Data Persistence

- Saves/loads user data to/from users.dat file
- Uses Java serialization for storage

2. Typing Test Module (4 Features)

1. Dynamic Text Generation

- Pulls random text based on selected difficulty level
- Supports three difficulty levels (easy/medium/hard)

2. Premium Difficulty Selection

- Allows premium users to choose difficulty
- Free users default to easy mode

3. Real-Time Typing GUI

- Color-coded character feedback (correct/incorrect)
- Live WPM/mistakes counter

4. Test Completion Handling

- Auto-submits when text is fully typed
- Manual submission via Enter key

3. Feedback and Result Module (4 Features)

1. Performance Metrics Calculation

- Computes WPM, accuracy, and mistake count
- Handles edge cases (zero-length input, etc.)

2. Visual Results Display

- Dedicated results screen with all metrics
- Clear formatting of numerical results

3. Result Storage

- Automatically saves to leaderboard

COMSATS UNIVERSITY ISLAMABAD

- Preserves username association

4. Navigation Control

- "Back to Dashboard" functionality
- Clean window management

4. User and Leaderboard Module (5 Features)

1. User Hierarchy System

- Abstract User base class
- FreeUser and PremiumUser implementations

2. Leaderboard Management

- Tracks top 10 global results
- Sorts entries by WPM (descending)

3. Personal History Tracking

- Stores all results per user
- Sortable history view

4. Premium Features Gate

- Export functionality limited to premium
- UI indicators for locked features

5. Data Persistence

- Saves/loads results to results.dat
- Serializable object storage

5. Text Generator Module (5 Features)

1. Multi-Difficulty Support

- Three text difficulty tiers
- Separate text files for each

2. File Loading System

- Reads from easy.txt, medium.txt, hard.txt
- Provides fallback default texts

3. Random Selection

- Randomly picks text from appropriate pool
- Uniform distribution across options

4. Error Handling

- Graceful fallback for missing files
- Input validation for difficulty levels

5. Dynamic Configuration

- Runtime difficulty switching
- Premium user customization

COMSATS UNIVERSITY ISLAMABAD

6. GUI and Event Handling Module (5 Features)

1. Unified Window Management

- Consistent sizing/positioning
- Proper dispose()/EXIT_ON_CLOSE usage

2. Interactive Components

- Buttons with action listeners
- Text fields with key listeners

3. Real-Time Feedback

- Live typing visualization
- Dynamic stats updating

4. Scrollable Content Areas

- JScrollPane for long texts/results
- Monospaced fonts for alignment

5. Modal Dialogs

- JOptionPane for alerts/inputs
- JFileChooser for export paths

COMSATS UNIVERSITY ISLAMABAD

Complete Project Walkthrough: Typing Speed Test Application

This project is a desktop application built with Java Swing for its Graphical User Interface (GUI), designed to test a user's typing speed and accuracy. It features user management (free vs. premium), a typing test, and a leaderboard/history system.

1. Application Startup: TypingApp.java (The Conductor)

- * What it is: This is the absolute starting point of entire application. Think of it as the main switch.

- * When you run `java TypingApp`, the main method is executed.

- * Inside main, the first thing that happens is the creation of a `Leaderboard<Result>` instance:

```
Leaderboard<Result> globalLeaderboard = new Leaderboard<>();
```

- * **Interaction:** This `globalLeaderboard` object will be passed around to different parts of the application (like `LoginScreen`, `DashboardScreen`, `TypingTest`, `ResultScreen`, `LeaderboardScreen`, `HistoryScreen`, `ExportScreen`). This ensures that everyone is working with the same set of test results. It's like having a single, central record book for all scores.

- * Next, a `LoginScreen` is created and displayed, and the `globalLeaderboard` instance is passed to it:

```
new LoginScreen(globalLeaderboard);
```

- * **Interaction:** This immediately transitions the user experience from a blank screen to the login interface.

2. User Authentication: LoginScreen.java (The Gatekeeper)

This is the first GUI the user sees. It handles user login and registration.

- * **Initialization:** When `LoginScreen` is created, its constructor receives the `globalLeaderboard` and stores it. It then immediately tries to `loadUsers()` from a file named `users.dat`.

- * If `users.dat` exists, it loads previously registered users into a static `ArrayList<User>` called `users`.

- * If `users.dat` doesn't exist (first run), it creates an empty `ArrayList`.

COMSATS UNIVERSITY ISLAMABAD

- * GUI Display: Text fields for username and password, along with "Login" and "Register" buttons, are displayed.

- * User Input: The user types their username and password.

- * "Login" Button Click:

- * The actionPerformed method for the login button is triggered.

- * It retrieves the entered username and password.

- * It iterates through the users ArrayList.

- * **Interaction (User.java):** For each User object in the list, it calls user.getUsername() to compare the username and user.checkPassword(pass) to verify the password. This is where polymorphism comes into play: both FreeUser and PremiumUser inherit getUsername() and checkPassword() from the User abstract class.

- * If a match is found:

- * A "Login successful!" message is shown.

- * The LoginScreen window is dispose()d (closed).

- * **Interaction (DashboardScreen.java):** A DashboardScreen is created, and the authenticated User object (e.g., FreeUser or PremiumUser) and the globalLeaderboard are passed to its constructor. This is crucial for maintaining the user's session and passing the central leaderboard.

- * If no match, "Invalid username or password!" is displayed.

- * "Register" Button Click:

- * The actionPerformed method for the register button is triggered.

- * It retrieves the entered username and password.

- * It checks if the username already exists in the users ArrayList.

- * If unique, it prompts the user to choose between "Free" or "Premium" account types using a JOptionPane.showOptionDialog.

COMSATS UNIVERSITY ISLAMABAD

* **Interaction (FreeUser.java, PremiumUser.java):** Based on the choice, either a FreeUser or PremiumUser object is instantiated with the provided credentials. This demonstrates polymorphism in action, creating the correct concrete user type.

* The newUser object is added to the users ArrayList.

* **Persistence (users.dat):** The saveUsers() method is called to serialize (convert to bytes) the entire users ArrayList and write it to users.dat file. This is how user data is saved permanently.

* A "User registered successfully!" message is displayed, and fields are cleared.

3. Main Navigation: DashboardScreen.java (The Hub)

This is the main screen after a successful login. It acts as the central hub from which the user can access different features of the application.

* **Initialization:** The constructor receives the currentUser (the logged-in User object) and the globalLeaderboard instance.

* **GUI Display:** A welcome message with the username is shown. Buttons for "Start Typing Test", "View Global Leaderboard", and "View My History" are always present.

* **Premium Feature:** An "Export My History" button is added only if currentUser.isPremium() returns true. This demonstrates a core premium feature and polymorphism, as the isPremium() method is defined in User and overridden by FreeUser (returns false) and PremiumUser (returns true).

* **Button Clicks (Event Handling):** Each button has an ActionListener that triggers specific actions:

* "Start Typing Test" Click:

* The DashboardScreen is dispose()d (closed).

* **Interaction (TypingTest.java):** A new TypingTest object is created, passing the currentUser and the globalLeaderboard instance. This initiates the typing test module.

* "View Global Leaderboard" Click:

* **Interaction (LeaderboardScreen.java):** A new LeaderboardScreen is created, passing only the globalLeaderboard instance. This opens a separate window showing top scores.

* "View My History" Click:

COMSATS UNIVERSITY ISLAMABAD

* **Interaction (HistoryScreen.java):** A new HistoryScreen is created, passing the globalLeaderboard instance and the currentUser.getUsername(). This opens a window showing the logged-in user's past results.

* "Export My History" Click (Premium):

* **Interaction (ExportScreen.java):** A new ExportScreen is created, passing the globalLeaderboard instance and the currentUser.getUsername(). This allows premium users to save their data to a file.

4. The Typing Test: TypingTest.java, TextGenerator.java, TypingTestGUI.java, FeedbackAnalyzer.java

This is the **core of the application** where the actual test happens.

* TypingTest.java (The Test Manager):

* Initialization: Its constructor receives the currentUser and globalLeaderboard. It also initializes TextGenerator and FeedbackAnalyzer.

* Difficulty Selection (selectLevelAndGetText()):

* If the currentUser is a FreeUser, difficulty is automatically set to "easy".

* If currentUser is a PremiumUser, a JOptionPane prompts them to choose "Easy" (1), "Medium" (2), or "Hard" (3).

* **Interaction (TextGenerator.java):** It calls textGenerator.setDifficulty() to inform the text generator of the chosen level.

* Text Retrieval (getText()):

* Interaction (TextGenerator.java): It calls textGenerator.getRandomText() to get a random passage based on the selected difficulty.

* Error handling: If no text is found (e.g., text files are empty), it shows an error and returns to the DashboardScreen.

* Launching GUI (startTypingTest()):

COMSATS UNIVERSITY ISLAMABAD

* **Interaction (TypingTestGUI.java):** It creates and displays a TypingTestGUI, passing the originalText to be typed and setting itself (this) as the TypingTestListener. This sets up a callback mechanism.

* Receiving Results (onTypingComplete() - Callback from GUI):

* This method is part of the TypingTestGUI.TypingTestListener interface, and it's called by TypingTestGUI when the user finishes typing or presses Enter.

* It receives the typedText and timeTakenSeconds.

* **Interaction (FeedbackAnalyzer.java):** It calls feedbackAnalyzer.evaluatePerformance(originalText, typedText, timeTakenSeconds) to calculate WPM, accuracy, and mistakes.

* **Interaction (Result.java, Leaderboard.java):** It calls storeResult():

* storeResult() creates a new Result object with the user's name, WPM, accuracy, and mistakes.

* It then calls leaderboard.addResult(result) to add this new result to the central globalLeaderboard instance, which handles saving it to results.dat.

* **Interaction (ResultScreen.java):** It calls displayResult() which creates and displays a ResultScreen, passing all the performance metrics, the currentUser, and globalLeaderboard for navigation.

* **TextGenerator.java (The Content Provider):**

Manages and provides the text passages for the typing test.

* **Initialization:** Loads text passages from easy.txt, medium.txt, and hard.txt files into separate ArrayLists. Includes fallback text if files are missing or empty.

* **setDifficulty(int level):** Sets an internal difficultyLevel string ("easy", "medium", "hard") based on the input number.

* **getRandomText():** Selects one of the three ArrayLists based on the current difficultyLevel and returns a random sentence from that list.

* **TypingTestGUI.java (The User Interface for Typing):**

* **What it does:** Provides the interactive GUI where the user types and receives real-time feedback.

COMSATS UNIVERSITY ISLAMABAD

- * showGUI(String textToType, TypingTestListener listener):

- * Sets up the Swing JFrame with a JTextPane to display the original text and a JTextField for user input.

- * Real-time Visual Feedback: As the user types, a KeyListener monitors each character:

- * It compares the typed character with the originalText character at the current index.

- * JTextPane Styling: It dynamically changes the color of the character in the JTextPane to blue for correct and red for incorrect using StyledDocument and Style objects. This is a very responsive visual cue.

- * Handles Backspace by moving currentIndex back and resetting the character's style to default.

- * Timer: A Swing Timer starts on the first key press. This timer updates a timerLabel every 100 milliseconds.

- * Real-time Stats: The timerLabel dynamically shows elapsed time, current WPM, and current Mistakes as the user types, giving constant feedback. This involves simple on-the-fly calculations (not using FeedbackAnalyzer until the end).

- * Test Completion:

- * When the user types all characters of originalText, or presses Enter at any point, the completeTest() method is called.

- * This stops the timer, calculates the final timeTaken, closes the TypingTestGUI window.

- * Callback: It then calls listener.onTypingComplete(typedText.toString(), timeTaken). This is the crucial callback that sends the results back to the TypingTest.java manager.

5. Results and Leaderboard: Result.java, Leaderboard.java, ResultScreen.java, LeaderboardScreen.java, HistoryScreen.java, ExportScreen.java

This module handles storing and displaying performance data.

- * **Result.java (The Data Structure):**

- * What it is: A simple Java Object that holds the data for a single typing test performance: username, WPM, accuracy, and mistakes.

COMSATS UNIVERSITY ISLAMABAD

* **Serializable:** It implements `Serializable`, which is essential because `Leaderboard.java` will write these `Result` objects directly to a binary file (`results.dat`).

* **Leaderboard.java (The Data Manager):**

* **What it is:** The central manager for all `Result` objects. It uses Java Generics (`Leaderboard<T extends Result>`) to make it flexible, though currently, it only handles `Result` objects.

* **Persistence (`results.dat`):**

* On initialization, it calls `loadResults()` to read all previously saved `Result` objects from `results.dat` into its internal `ArrayList<T> results`.

* Whenever `addResult(T result)` is called, it immediately calls `saveResults()` to write the entire updated `ArrayList` back to `results.dat`. This ensures data is always saved after a new result.

* `addResult(T result)`: Adds a new `Result` to the list.

* `getTopResults(int topN)`: Sorts the results `ArrayList` by WPM in descending order and returns the top N results.

* `getUserHistory(String username)`: Filters the results `ArrayList` to find all entries for a specific username and returns them as a list.

* **ResultScreen.java (The Immediate Feedback):**

Displays the results of the just completed typing test.

* **Initialization:** Receives WPM, accuracy, mistakes, the `currentUser`, and `globalLeaderboard`.

* **GUI Display:** Shows the WPM, accuracy, and mistake count for the session.

* **"Back to Dashboard" Button:** Allows the user to navigate back to the `DashboardScreen`, passing the `currentUser` and `globalLeaderboard` to maintain context.

* **Crucial Integration:** It explicitly calls `leaderboard.addResult(new Result(user.getUsername(), wpm, accuracy, mistakes))` again. While `TypingTest` already added the result, this ensures the result is definitely added even if there's a slight timing difference in the previous call or if this screen is opened in a different context (though in our current flow, it's typically redundant but harmless). The `JOptionPane` confirming "Result Saved" reinforces this.

* **LeaderboardScreen.java (The Global Ranking):**

COMSATS UNIVERSITY ISLAMABAD

Displays the overall top performers from all users.

- * Initialization: Receives the globalLeaderboard instance.

- * GUI Display: Uses a JTextArea to format and display the top 10 (or fewer) results, including Rank, Username, WPM, Accuracy, and Mistakes, retrieved via `leaderboard.getTopResults(TOP_N)`.

- * **HistoryScreen.java (Personal Performance Tracking):**

- * What it does: Displays the historical typing performance for the currently logged-in user.

- * Initialization: Receives the globalLeaderboard and the username.

- * GUI Display: Uses a JTextArea to display all results associated with the username, retrieved via `leaderboard.getUserHistory(username)`. Results are sorted by WPM for display.

- * **ExportScreen.java (Premium Data Export):**

- * What it does: Allows premium users to export their typing history to a text file.

- * Initialization: Receives the globalLeaderboard and the username.

- * GUI Display: A button labeled "Export History (TXT)".

- * "Export History" Button Click:

- * Uses JFileChooser to allow the user to select a location and filename for the export.

- * File Writing: Retrieves the user's history using `leaderboard.getUserHistory(username)`.

- * Uses FileWriter to write the formatted history data into the chosen text file.

- * Provides success or error messages using JOptionPane.



END !